Prérequis

Avoir terminé le laboratoire précédent (ou utiliser sa correction). N'oubliez pas d'importer la base de données fournie.

Upload de fichier

Vous allez maintenant gérer la partie *upload* de l'image de profil.

Information

Lors de la soumission du formulaire, si celui-ci contient un fichier (élément `<input type="file">`), le fichier a été envoyé sur le serveur dans un dossier temporaire. Pour déplacer ce fichier dans un dossier permanent, on peut utiliser la fonction `move_uploaded_file`.

Avant de déplacer le fichier, il convient de vérifier qu'il correspond à ce que l'on attend : le tableau superglobal $_FILES[1] contient les informations sur le ou les fichiers *uploadés*.

- `$_FILES['nom_du_champ_file']` contient un tableau associatif avec les informations sur le fichier *uploadé*. `nom_du_champ_file` est le nom donné à l'`<input type="file">` du formulaire :
- `$_FILES['nom_du_champ_file']['name']` contient le nom du fichier
- `$_FILES['nom_du_champ_file']['type']` contient le type (*media type*) du fichier[2] (par exemple `image/gif`, `image/png`, `image/jpeg`, ...)
- `$_FILES['nom_du_champ_file']['size']` contient la taille en octets
- `$_FILES['nom_du_champ_file']['tmp_name']` contient l'emplacement temporaire du fichier
- `$_FILES['nom_du_champ_file']['error']` contient éventuellement un code d'erreur (`0` si pas d'erreur).

Commençons par vérifier qu'une image a été *uploadée* :

```
if(isset($_FILES['image']['name']) && $_FILES['image']['name'] != ''){
    if($_FILES['image']['error']==0){
        //...
    } else {
        $error = "Error while uploading file.";
    }
}
```

Remarque

La variable `$error` peut être affichée de la même manière que la variable `$success` utilisée plus haut.

Ensuite, il faut vérifier que le type du fichier reçu fait partie des types autorisés (variable `$typeOK`). En même temps, il faut prévoir un nouveau nom de fichier en utilisant le nom de l'utilisateur afin d'assurer que deux images n'auront jamais le même nom (variable `$saveTo`) :

```
if(isset($_FILES['image']['name']) && $_FILES['image']['name'] != ''){
    if($_FILES['image']['error']==0){

        $typeOK = TRUE;

        if($_FILES['image']['type']=="image/gif")
            $saveTo = $user.".gif";
        else if($_FILES['image']['type']=="image/jpeg")
            $saveTo = $user.".jpg";
        else if($_FILES['image']['type']=="image/png")
            $saveTo = $user.".png";
        else {
            $typeOK = FALSE;
            $error = "Unsupported image format: gif, jpeg or png!";
        }

    } else {
        $error = "Error while uploading file.";
    }
}
```

Si le type est vérifié, on peut enfin déplacer l'image dans notre espace du serveur Web et mettre à jour la base de données (la colonne `picture_path` du membre connecté) :

```
if(isset($_FILES['image']['name']) && $_FILES['image']['name'] != ''){
    if($_FILES['image']['error']==0){

        $typeOK = TRUE;

        if($_FILES['image']['type']=="image/gif")
            $saveTo = $user.".gif";
        else if($_FILES['image']['type']=="image/jpeg")
            $saveTo = $user.".jpg";
        else if($_FILES['image']['type']=="image/png")
            $saveTo = $user.".png";
        else {
            $typeOK = FALSE;
            $error = "Unsupported image format: gif, jpeg or png!";
        }

        if($typeOK){
            move_uploaded_file($_FILES['image']['tmp_name'], $saveTo);
            try{
                $query = $pdo->prepare("UPDATE Members SET picture_path=:path
WHERE pseudo=:pseudo ");
                $query->execute(["path"=>$saveTo, "pseudo"=>$user]);
                $success = "Your profile has been successfully updated.";
                $picture_path = $saveTo; //pour que l'image se mette à jour
dans l'HTML ci-dessous.
            } catch (Exception $ex) {
```

```
                die("Error while accessing database. Please contact your
administrator.");
            }
        }
    } else {
        $error = "Error while uploading file.";
    }
}
```

---

1. Source et plus d'infos sur : https://openclassrooms.com/courses/concevez-votre-site-web-avec-php-et-mysql/transmettre-des-donnees-avec-les-formulaires#/id/r-913098 ↩
2. Voir la liste complète ici : https://www.iana.org/assignments/media-types/media-types.xhtml ↩

Hachage du mot de passe

Pour des raisons de sécurité évidentes, il ne faut jamais stocker le mot de passe des utilisateurs. Imaginez ce qu'il pourrait se passer si un pirate arrivait à accéder à votre base de données...

Un premier pas vers plus de sécurité est de ne pas stocker le mot de passe mais son empreinte (un hachage ou *hash*).

1. A l'aide de PhpMyAdmin, supprimez de la base de données les utilisateurs existants : leur mot de passe est stocké en clair.
2. Modifiez le fichier `signup.php` afin de ne pas stocker le mot de passe en clair mais de stocker son empreinte. Pour calculer l'empreinte, utilisez la fonction PHP `password_hash($password, PASSWORD_BCRYPT);`
3. A l'aide de votre page `signup.php`, ajoutez les utilisateurs `ben` (mot de passe `ben`) et `admin` (mot de passe `admin`).
4. Constatez, à l'aide de phpMyAdmin, que les mots de passes ne sont plus stockés en clair.
5. Modifiez le fichier `login.php` afin de comparer le mot de passe reçu du formulaire avec l'empreinte contenue dans la base de données. Utilisez pour ce faire la fonction PHP `password_verify($password, $hash)`. Les utilisateurs `ben` et `admin` devraient maintenant pouvoir se connecter.

Page d'erreur

Afin de pouvoir afficher les erreurs critiques de façon personnalisée (plutôt que d'afficher uniquement la chaîne de caractères passée à la fonction `die`), nous allons créer une page d'erreur (`error.php`) :

```
<!DOCTYPE html>
<html>
    <head>
        <title>Error</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <link href="styles.css" rel="stylesheet" type="text/css"/>
    </head>
    <body>
        <div class="title">Error</div>
        <div class="main">
            <div class="errors"><?= escape($error) ?></div>
        </div>
    </body>
</html>
```

Plutôt que d'appeler `die()` lors d'une erreur critique, vous appellerez une fonction `abort()` définie de la façon suivante :

```
function abort($err)
{
    global $error;
    $error = $err;
    include 'error.php';
    die;
}
```

Modifiez votre code afin d'utiliser cette fonction systématiquement lorsque vous utilisiez `die()`. Vérifiez son fonctionnement.

Refactorisations

Eviter la redondance de code

Beaucoup de pages contiennent la même partie de code : l'affichage du menu (Home, Members, Friends, ...).

Arrangez-vous pour que ce code ne soit pas copié collé dans toutes ces pages à l'aide de la fonction `include()`.

Une fonction pour se connecter à la base de données

Dans le fichier `functions.php`, le code PHP qui permet de se connecter à la base de données et d'obtenir la variable `$pdo` devrait être dans une fonction `connect()` qui renvoie cet objet `PDO`. Créez une telle fonction et utilisez-la aux endroits nécessaires.

Tous les accès à la base de données dans des fonctions

Factorisez tout le code d'accès à la base de données dans des fonctions que vous placerez dans le fichier `functions.php`.

Pour illustrer cette démarche, voici un exemple concret.

Dans la page `signup.php` du tuto MSN, on trouve le code suivant qui vérifie si le pseudo encodé n'est pas déjà utilisé par un membre :

```
...
$pdo = connect();
...
try{
    $query = $pdo->prepare("SELECT * FROM Members WHERE pseudo=:pseudo");
    $query->execute(["pseudo"=>$pseudo]);
    $result = $query->fetchAll();
    if(count($result) != 0)
        $errors[] = "This username is already taken";
} catch (Exception $e){
    abort("Error while accessing database. Please contact your
administrator.");
}
...
```

L'idée est d'extraire ce code de la page et d'en faire une fonction dont le but est de rendre le service de vérification qu'un pseudo n'est pas déjà utilisé. On peut par exemple créer la fonction suivante qui retourne un booléen :

```
function is_pseudo_available($pseudo) {
    $pdo = connect();
    try{
        $query = $pdo->prepare("SELECT * FROM Members WHERE pseudo=:pseudo");
        $query->execute(["pseudo"=>$pseudo]);
        $result = $query->fetchAll();
        return count($result) === 0;
    } catch (Exception $e){
        abort("Error while accessing database. Please contact your
administrator.");
    }
}
```

Du coup, le code de la page `signup.php` devient :

```
...
if (!is_pseudo_available($pseudo))
    $errors[] = "This username is already taken";
...
```

Les principaux avantages de cette approche sont :

1. la lisibilité du code des pages desquelles on a extrait le code (parfois verbeux) d'accès à la base de données ;
2. la ré-utilisabilité de ces fonctions d'accès à la base de données : si plusieurs pages ont besoin du même "service", elles se contentent d'appeler la même fonction, alors qu'auparavant nous aurions dupliqué le code.